Algorithmentechnik

§3 Greedy-Algorithmen

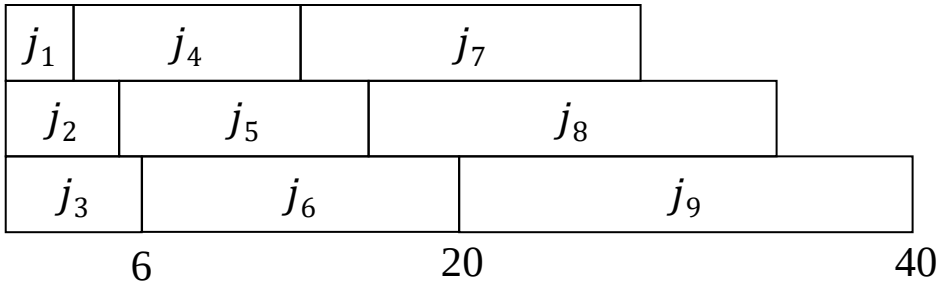
Prof. Dr. Georg Umlauf

Lösung für (1): Shortest-Job-First

- Wähle einen noch nicht bedienten Job mit kürzester Ausführungszeit und weise ihm denjenigen Prozessor zu, der als nächster einen Job bedienen kann.

Beispiel:

- Zuteilung von 9 Jobs auf 3 Prozessoren mit dem Shortest-Job-First-Algorithmus.



Job	Ausführungs-zeit	Abschluss-zeit
j_1	3	3
j_2	5	5
j_3	6	6
j_4	10	13
j_5	11	16
j_6	14	20
j_7	15	28
j_8	18	34
j_9	20	40

Minimierung der durchschnittlichen Abschlusszeit

- Begründung für mehrere Prozessoren, d.h. $p \geq 1$:

  Im Beispiel ist die durchschnittliche Abschlusszeit:

  $$[\, t_1 + t_2 + t_3 + (t_1 + t_4) + (t_2 + t_5) + (t_3 + t_6) + (t_1 + t_4 + t_7) + (t_2 + t_5 + t_8) + (t_3 + t_6 + t_9)\,]/9$$

  $$= [3t_1 + 3t_2 + 3t_3 + 2t_4 + 2t_5 + 2t_6 + t_7 + t_8 + t_9]/9$$

  - Dieser Wert ist minimal, wenn $t_1, t_2, t_3 \leq t_4, t_5\,, t_6 \leq t_7, t_8, t_9$.
  - Das wird jedoch vom Algorithmus gewährleistet.

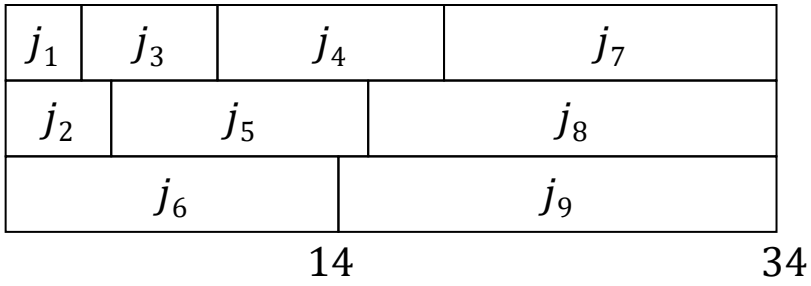
Analyse

- Im Algorithmus müssen alle Prozesse initial einmal sortiert werden und alle weiteren Schritte haben linear Laufzeit:

  $$O(n \log n).$$

Minimierung der gesamten Abschlusszeit (2)

- Der Shortest-Job-First-Algorithmus liefert im allgemeinen nur eine suboptimale Lösung.
  - Im Beispiel ist die gesamte Abschlusszeit $= 40$.
  - Es kann aber eine gesamte Abschlusszeit von 34 erreicht werden.



- Das Problem der Minimierung der gesamten Abschlusszeit beim Job-Scheduling ist NP-vollständig.

§3.3 Huffman-Codes

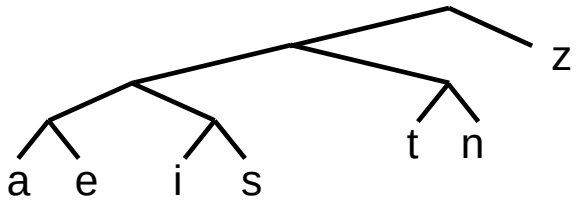
Fano-Bedingung (Präfix-Bedingung)

- Ein Code erfüllt die Fano-Bedingung, falls keine Codierung eines Zeichens Präfix (Anfangsstück) einer Codierung eines anderen Zeichens ist.
- Durch Code-Bäume dargestellte Codes erfüllen immer die Fano-Bedingung.
- Codes, die die Fano-Bedingung erfüllen, sind eindeutig decodierbar.

Beispiel

- Decodiere die folgende Binärfolge mit dem Binärcode:

0010110110100010



Zeichen	Code
a	0000
e	0001
i	0010
s	0011
t	010
n	011
z	1

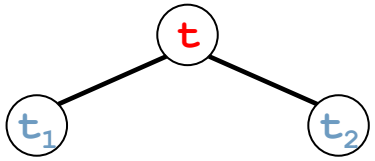
Huffman-Coding

Verwende einen Binärcode variabler Länge, wobei häufigere Zeichen eine kürzere Codierung erhalten.

(1) - Beginne mit einem Wald von Code-Bäumen, wobei jeder Baum anfangs aus genau einem Zeichen besteht.
- Jeder Baum besitzt eine Wahrscheinlichkeit p, die die Summe der Wahrscheinlichkeit der Zeichen im Baum ist.

(2) Greedy-Schritt:
Wähle die beiden Bäume t_1 und t_2 mit der kleinsten Wahrscheinlichkeiten p_1 und p_2 und bilde daraus einen neuen Baum t mit der Wahrscheinlichkeit $p = p_1 + p_2$:

Greedy-Schritt.



(3) - Wiederhole Schritt (2) solange, bis nur noch ein Baum vorhanden ist.
- Dieser Baum ist dann der gesuchte optimale Binärcode.

§3.3 Huffman-Codes

Beispiel:

Zeichen	Häufigkeit
a	10
e	15
i	12
s	3
t	4
n	13
z	1

- Schritt `(1)`:
  Wald von Bäumen mit jeweils genau einem Zeichen und Häufigkeiten als Gewichte (blau)

  a)

10	15	12	3	4	13	1
a	e	i	s	t	n	z

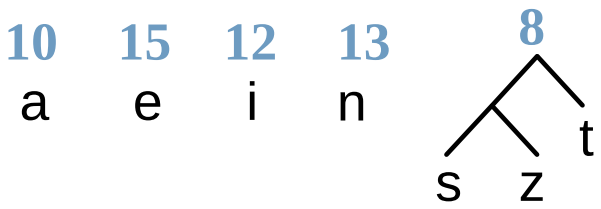
- Schritt `(2)` wiederholt angewendet:
  Baue die beiden Bäume mit geringster Häufigkeit zu einem Baum zusammen.

  b)

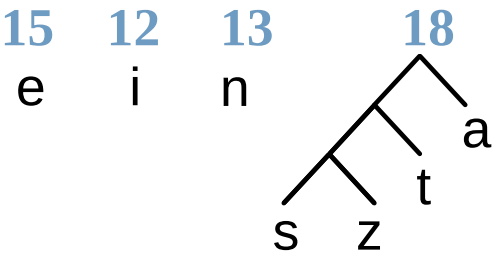
10	15	12	4	13	4
a	e	i	t	n	s z

§3.3 Huffman-Codes

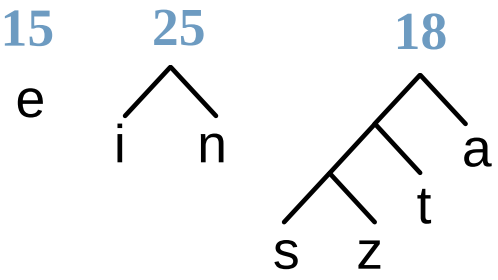
c)



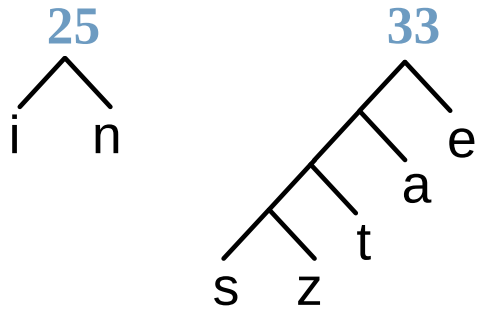
d)



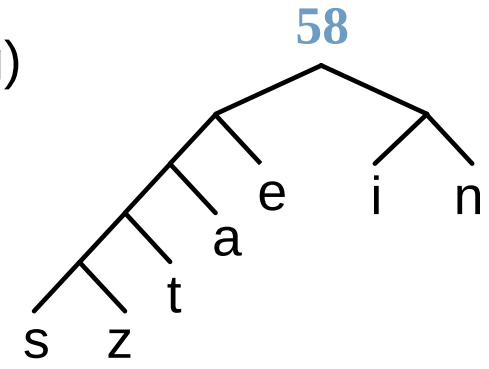
e)



f)



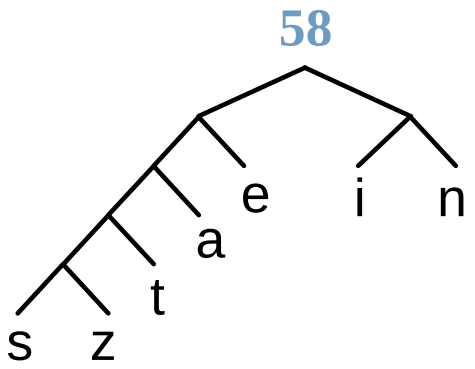
g)



h) fertig!

§3.3 Huffman-Codes

Optimaler Code:



Zeichen	Häufigkeit	Code
a	10	001
e	15	01
i	12	10
s	3	00000
t	4	0001
n	13	11
z	1	00001

Bemerkung

- Verwendet man diesen optimalen Code, benötigt man für die oben angegebenen Häufigkeiten:

  $10 \cdot 3 + 15 \cdot 2 + 12 \cdot 2 + 3 \cdot 5 + 4 \cdot 4 + 13 \cdot 2 + 1 \cdot 5 = 146$ Bit

  - Damit benötigt man $146/58 = 2.5$ Bit/Zeichen.
  - Im Vergleich dazu benötigt man beim Binärcode mit konstanter Bitlänge 3 Bit/Zeichen bzw. $58 \cdot 3 = 175$ Bit.

Analyse

- Zwei binäre Bäume können mit Laufzeit $O(1)$ vereinigt werden.
- Zum Auffinden der Teilbäume mit dem minimalen Gewicht, werden sie in einer Priority-Queue einsortiert.
  - Realisierung als Min-Heap mit dem Gewicht als Priorität, siehe Appendix.
- Der Aufbau einer Priority-Queue als Heap hat Laufzeit $O(n)$, siehe Appendix.
- Ein Element kann mit Laufzeit $O(\log n)$ aus der Priority-Queue entfernt werden.
- Gesamte Laufzeit für n Zeichen: $O(n \log n)$.

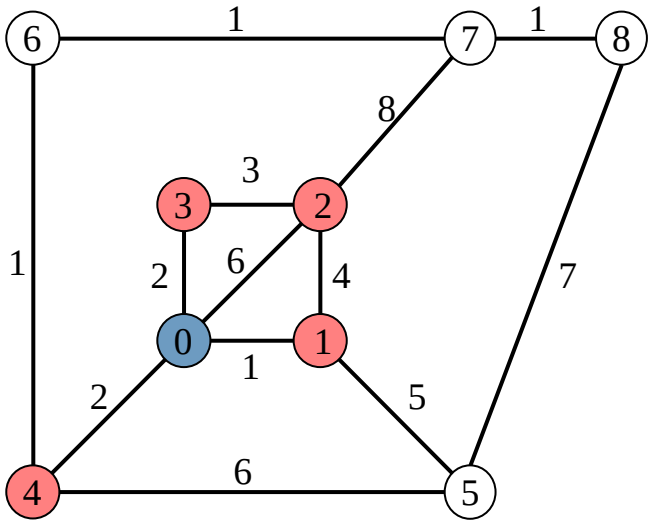
Optimalitätsbeweis?

Bemerkung

- Die Häufigkeitsanalyse kann auch einmalig (statisch) erfolgen.
  - Z.B. kommt in typisch deutschen Texten der Buchstabe „e“ (ohne Groß/Kleinschreibung) mit der Häufigkeit 0.175 vor.
  - Nimmt man einen Code mit konstanter Bit-Länge, dann benötigt man $\lceil \log_2 26 \rceil = 5$ Bit/Zeichen.
  - Mit einer Huffman-Codierung erhält man etwa 4.1 Bit/Zeichen.
  - Verbessern lässt sich das Verfahren, indem Buchstabenpaare oder noch längere Buchstabenfolgen betrachtet werden.
  - Bei Buchstabenpaaren erreicht man etwa 3.5 Bit/Zeichen.
- Einsatzgebiet: jpeg, zip, gzip, mp3, mpeg, ...

§3.4 Algorithmus von Dijkstra

Idee…



Kandidatenliste

Bereits besucht

d[v] Distanz zwischen v und Startknoten **s = 0**

p[v] Vorgängerknoten im kürzesten Weg nach **v**

v	0	1	2	3	4	5	6	7	8
d[v]	0	1	6	2	2	∞	∞	∞	∞
p[v]	-	0	0	0	0	-	-	-	-

Als nächstes wird Knoten 1 mit Distanzwert 1 besucht.

§3.4 Algorithmus von Dijkstra

```
void shortestPath( DistanzGraph G, Vertex s,
                   int d[], Vertex p[])
{
    for ( jeden Knoten v ) { d[v]=∞; p[v]=undef.; }
    d[s] = 0;
(1) Set<Vertex> K; K.insert(s);
    while ( !K.empty() ) {
(2)   Lösche Knoten v aus K mit minimalem d[v];
      for ( jeden Nachbarn w von v ) {
(3)     if ( d[w]==∞ ) K.insert(w);
(4)     if ( d[v]+c(v,w)<d[w] ) {
(5)       d[w]=d[v]+c(v,w); p[w]=v;
        }
      }
    }
}
```

Eingabe

Ausgabe

In der Kandidatenliste sind alle Knoten abgespeichert, die als nächstes besucht werden müssen.

Greedy-Schritt.

Distanzwert für **w** verbessert sich. Kürzester Weg nach **w** geht nun über **v**.

Nur für Knoten **w** aus K kann der Distanzwert verbessern werden.

§3.4 Algorithmus von Dijkstra

Kandidatenliste als Vorrangwarteschlange (priority queue)

Vorrangwarteschlangen bieten effiziente Realisierungen der Operationen:

- **`insert(v,d)`**: Fügt Element mit Vorrangwert (Distanz) **`d`** ein. Aufruf in `(1)` und `(3)`.
- **`v = delMin()`**: Löscht Element mit kleinstem Vorrangwert **`d`**. Aufruf in `(2)`.
- **`change(v,dNeu)`**: Findet und ändert den Vorrangwert **`d`** von **`v`** auf **`dNeu`**. Aufruf in `(5)`.
- Vorrangwarteschlangen können mit Heaps (s. Appendix) implementiert werden.
  - Damit können **`insert`**, **`delMin`** und das Ändern bei **`change`** in $O(\log n)$ durchgeführt werden.
  - Für **`change`** wird noch zusätzlich eine Suchstruktur (z.B. AVL) benötigt, um den Knoten **`v`** in $O(\log n)$ finden zu können.

Kandidatenliste als Vorrangwarteschlange

Analyse

- Die Initialisierung benötigt $O(|V|)$.
- Die Operationen in `(3)` und `(5)` benötigen $O(\log|V|)$.
- Jede Kante im Graph wird genau einmal in einer der `for`-Schleifen betrachtet.

▶ Damit ist für alle `for`-Schleifen insgesamt $O(|E| \cdot \log|V|)$ notwendig.

- Dazu kommt noch $|V|$-mal die Ausführung von `(2)`, die in $O(\log|V|)$ realisiert werden kann.

▶ Insgesamt: $O\big((|E| + |V|) \cdot \log|V|\big)$.

§3.4 Algorithmus von Dijkstra

Kandidatenliste als einfaches, unsortiertes Feld

- **`insert(v,d)`**: Aufwand für das Einfügen $O(1)$.
- **`v = delMin()`**: Löschen des kleinsten Elements in $O(n)$.
- **`change(v,dNeu)`**: Zum Ändern des Vorrangwerts ist keine Umorganisation des Feldes notwendig, daher Aufwand von $O(1)$.

Analyse

- Die `while`-Schleife wird $|V|$-mal durchgeführt.
  - Für `(2)` ist $O(|V|)$ notwendig.
- Die `for`-Schleife braucht insgesamt höchstens $O(|E|)$.
- Da jeder Knoten maximal einmal eingefügt wird, ergibt sich insgesamt:

$$O\left(|E| + |V|^2\right).$$

§3.4 Algorithmus von Dijkstra

Fazit

- Ist der Graph dünn besetzt, d.h. $|E| = O(|V|)$, dann ist die Variante als Vorrangwarteschlange besser.
- Ist der Graph dicht besetzt, d.h. $|E| = O(|V|^2)$, dann ist die Variante als einfaches Feld besser.

§3.4 Algorithmus von Dijkstra

Verbesserungen

- Bidirektionale Suche.
- A*-Algorithmus: Heuristik, um in eine bestimmte Zielrichtung zu gehen.
  - Euklidische Distanz oder vorberechnete Potentialfunktion.
- ALT-Algorithmus (A*-landmark-triangulation): anhand vorberechneter Orientierungspunkte die Distanz zum Ziel schätzt.
- ArcFlags: Aufteilung in Regionen und Vorberechnung der kürzesten Wege durch die Regionen.
- Berechnungszeiten bei 46 Mio Kanten und 18 Mio Knoten (europäisches Straßennetz)
  - Vorberechnung: Mehrere Tage
  - Berechnung der kürzesten Wege: 0.1 Sekunden

§3.5 Algorithmen von Prim und Kruskal

Problem

- Eingabe: Gewichteter Graph.
- Ausgabe: Minimal aufspannender Baum.

Typische Anwendung

- Zwischen n Orten soll ein Versorgungsnetz (Kommunikation, Strom, Wasser, etc.) aufgebaut werden, so dass je 2 Orte direkt oder indirekt miteinander verbunden sind.
  Die Verbindungskosten zwischen 2 Orten sind bekannt.
- Gesucht ist ein Versorgungsnetz mit den geringsten Kosten.

§3.5 Algorithmen von Prim und Kruskal

Definition

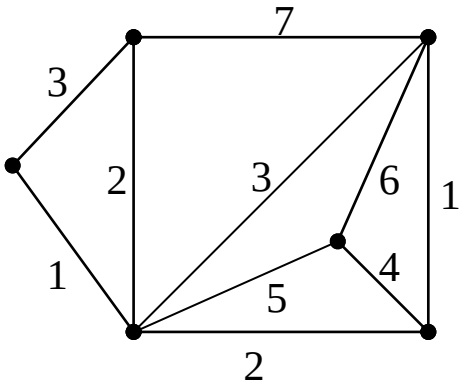
Sei $G = (V, E)$ ein ungerichteter, gewichteter Graph.

Dann heißt B ein **minimal aufspannender Baum**, falls B folgende Eigenschaften erfüllt:

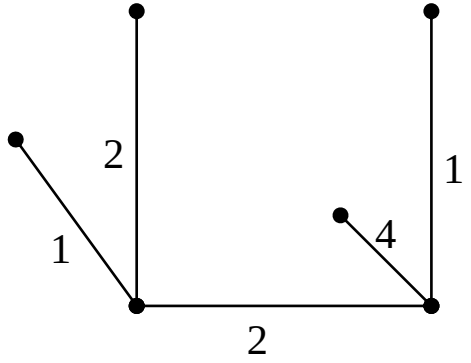
1. B ist ein Teilgraph von G mit gleicher Knotenmenge.
   - $B = (V, E')$ mit $E' \subseteq E$.
2. B ist ein Baum.
   - Ein azyklischer, ungerichteter Graph.
3. Die Summe der Kantengewichte von B ist minimal.
   - Es gibt keinen anderen Baum, der 1. und 2. erfüllt und eine kleinere Kantengewichtssumme hat.

§3.5 Algorithmen von Prim und Kruskal

Beispiel



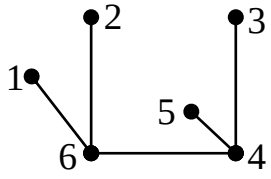
Ungerichteter Graph G

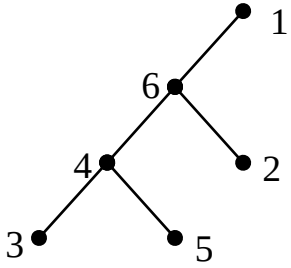


Minimal aufspannender Baum von G

Bemerkungen

- Ein azyklischer, ungerichteter Graph ist ein Baum: Wähle einen beliebigen Knoten als Wurzel und „verschiebe die anderen Knoten nach unten“.





Bemerkungen

- Bedeutung von „aufspannend“:

  Da ein minimal aufspannender Baum B zu einem Graphen G alle Knoten von G enthält und außerdem ein Baum zusammenhängend ist, spannt B den Graphen G auf.

- Der minimal aufspannende Baum muss nicht eindeutig sein.

§3.5 Algorithmen von Prim und Kruskal

Problem

- Eingabe: Gewichteter Graph.
- Ausgabe: Minimal aufspannender Baum.

Idee

- Der **Algorithmus von Prim** ist eine leichte Modifikation des Algorithmus von Dijkstra:
  - Beim Algorithmus von Dijkstra wird immer der Knoten mit der kürzesten Distanz zum Startknoten **`s`** als nächster besucht.
  - Beim Algorithmus von Prim wird immer der Knoten mit der billigsten Kante als nächster besucht.

Übung

§3.5 Algorithmen von Prim und Kruskal

Idee

- Zu jedem Zeitpunkt bilden die bereits besuchten Knoten einen minimal aufspannenden Baum.
  - Anfangs ist noch kein Knoten besucht.
  - Der bisher gefundene Baum wird wie beim Algorithmus für kürzeste Wege in einem Vorgängerfeld gehalten.
- Alle Knoten, die als nächstes besucht werden können, werden in einer Kandidatenliste gehalten.
  - Zu Anfang wird mit einem beliebigen Knoten als Kandidat begonnen.
  - Von den Kandidaten wird derjenige Knoten als nächster besucht, der… der über die billigste Kante zu erreichen ist.

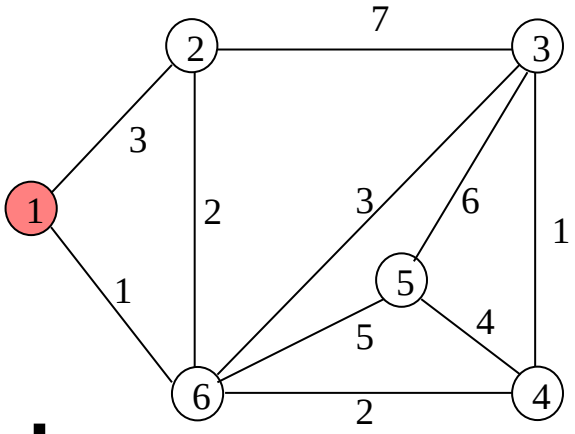
Übung

Beispiel:

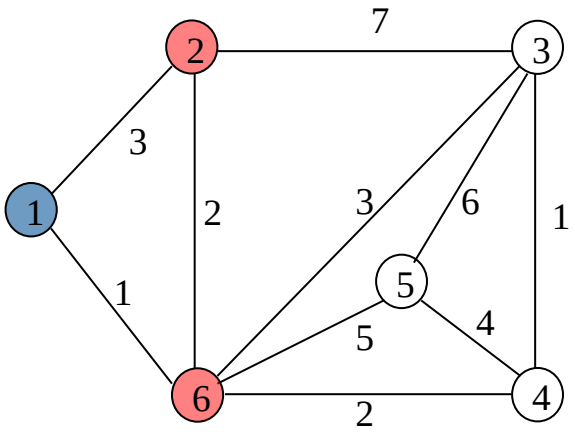
- Zu Beginn ist Knoten 1 einziger Kandidat.

Kandidaten

bereits besucht



- Knoten 1 wird besucht.
- Alle Nachbarn von Knoten 1 werden zu neuen Kandidaten.
- Blaue Knoten bilden den bereits gefundenen minimalen aufspannenden Baum.

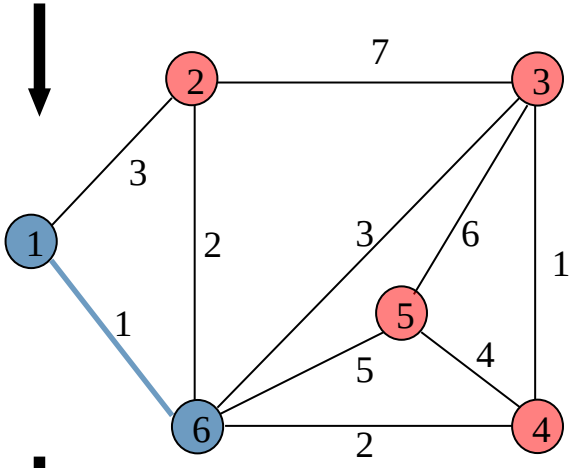


Knoten v	1	2	3	4	5	6
Gewichtsfeld d[v]	0	3	∞	∞	∞	1
Vorgängerfeld p[v]	-	1	-	-	-	1

§3.5 Algorithmen von Prim und Kruskal

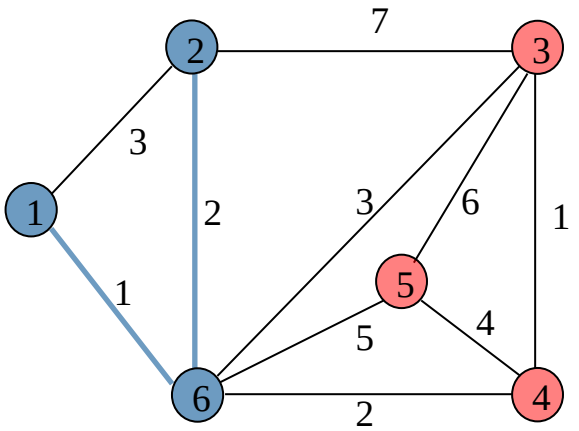
- Knoten 6 wird besucht, weil über günstigste Kante erreichbar.

Knoten v	1	2	3	4	5	6
Gewichtsfeld d[v]	0	2	3	2	5	1
Vorgängerfeld p[v]	-	6	6	6	6	1



- Knoten 2 wird besucht.
- Alternativ hätte man auch Knoten 4 nehmen können.

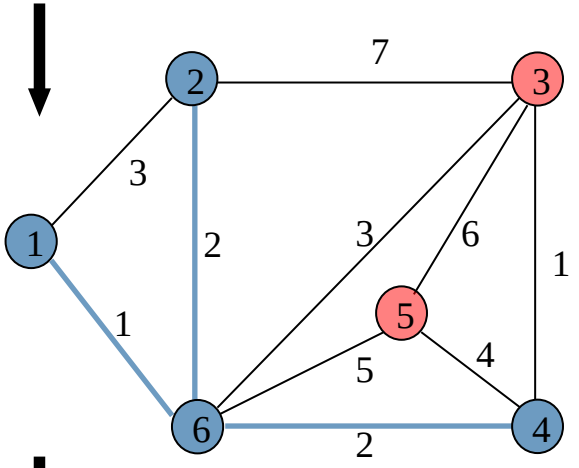
Knoten v	1	2	3	4	5	6
Gewichtsfeld d[v]	0	2	3	2	5	1
Vorgängerfeld p[v]	-	6	6	6	6	1



§3.5 Algorithmen von Prim und Kruskal

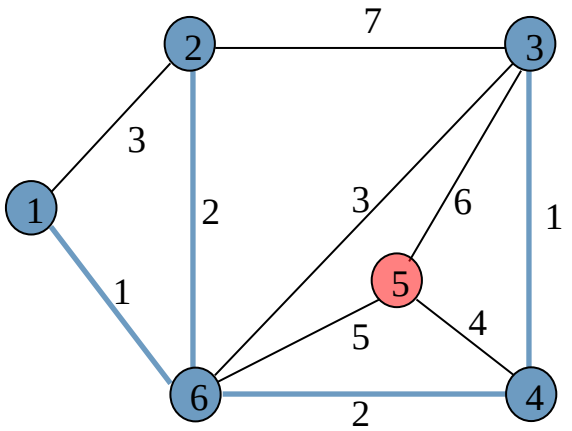
- Knoten 4 wird besucht.

Knoten v	1	2	3	4	5	6
Gewichtsfeld d[v]	0	2	1	2	4	1
Vorgängerfeld p[v]	-	6	4	6	4	1



- Knoten 3 wird besucht.

Knoten v	1	2	3	4	5	6
Gewichtsfeld d[v]	0	2	1	2	4	1
Vorgängerfeld p[v]	-	6	4	6	4	1



§3.5 Algorithmen von Prim und Kruskal

Eingabe

Ausgabe

```
void minimalSpanningTree(Graph G, Vertex p[])
{
   for ( jeden Knoten v ) {
      p[v]=undef.; imBaum[v]=false; d[v]=∞;
   }
   s = 1; d[s] = 0;                    // Startknoten
   Set<Vertex> K; K.insert(s); // Kandidatenliste
   while ( !K.empty() ) {
      Lösche Knoten v aus K mit minimalem d[v];
      imBaum[v] = true;                // v wird im Baum eingebaut
      for ( jeden Nachbarn w von v )
         if ( !imBaum[w] ) {
            if ( d[w]==∞ ) K.insert(w);
            if ( c(v,w)<d[w] ) {d[w]=c(v,w); p[w]=v; }
         }
   }
}
```

`d[w]` gibt für jeden Kandidat das Gewicht der billigsten Kante an, mit der er von den bereits besuchten Knoten erreicht werden kann.

Greedy-Schritt.

Falls nicht alle Knoten besucht wurden, existiert kein aufspannender Baum. Es müsste dann noch eine entsprechende Ausgabe erfolgen.

`d[w]` verbessert sich.

Übung

§3.5 Algorithmen von Prim und Kruskal

Analyse

Mit der gleichen Begründung wie beim Algorithmus von Dijkstra gilt:

a) Kandidatenliste als Vorrangwarteschlange

$$T = O((|E| + |V|) \cdot \log |V|).$$

b) Kandidatenliste als einfaches (unsortiertes) Feld

$$T = O(|E| + |V|^2).$$

Fazit

- Ist der Graph dünn besetzt, d.h. $|E| = O(|V|)$, dann ist die Variante a) als Vorrangwarteschlange besser.
- Ist der Graph dicht besetzt, d.h. $|E| = O(|V|^2)$, dann ist die Variante b) als einfaches Feld besser.

Übung

§3.5 Algorithmen von Prim und Kruskal

Problem

- Eingabe: Gewichteter Graph.
- Ausgabe: Minimal aufspannender Baum.

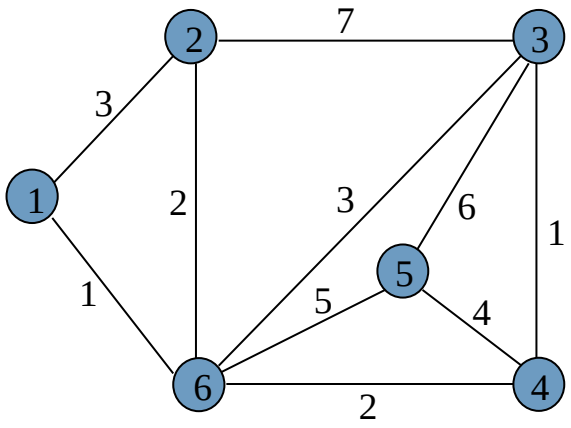
Idee

- Verwalte **Wald** (Menge von Bäumen) minimal aufspannender Bäume.
- **Start:** Wald aller Bäumen mit genau einem Knoten.
- **Vereinigungsschritt:**
  1. Suche die billigste Kante **`k`**, die zwei Bäume aus dem Wald verbindet.
  2. Vereinige die Bäume über **`k`** zu einem Baum.
- Es werden solange Vereinigungsschritte durchgeführt, bis nur noch ein Baum übrig bleibt.
- ▶ Das ist der minimal aufspannende Baum.

§3.5 Algorithmen von Prim und Kruskal

Beispiel

Start: Wald besteht aus **sechs** Bäumen mit jeweils genau einem Knoten.

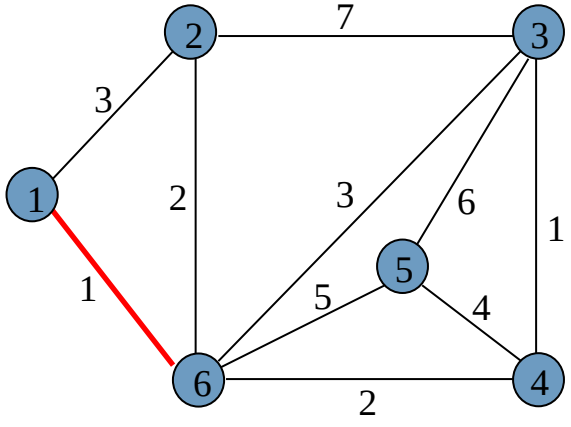


Vereinigungsschritt:

1. Kante $\mathbf{k} = (1,6)$ mit Gewicht 1 wird gewählt.

2. Die beiden Bäumen, die aus dem Knoten 1 und 6 bestehen, werden zu einem Baum vereinigt.

► Der Wald besteht nun noch aus **fünf** Bäumen.

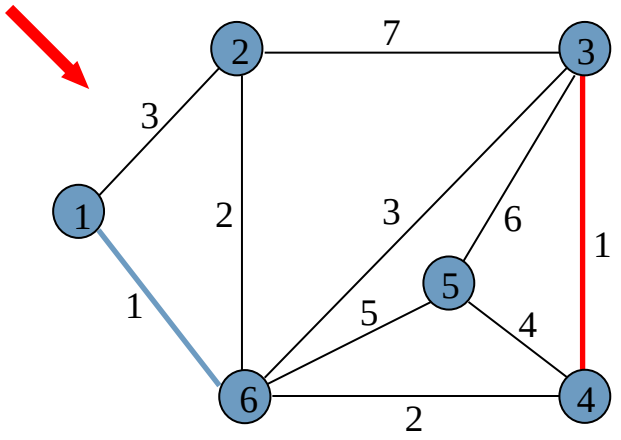


§3.5 Algorithmen von Prim und Kruskal

Vereinigungsschritt:

1. Kante $\mathbf{k} = (3,4)$ mit Gewicht 1 wird gewählt.
2. Die beiden Bäumen, die aus dem Knoten 3 und 4 bestehen, werden zu einem Baum vereinigt.

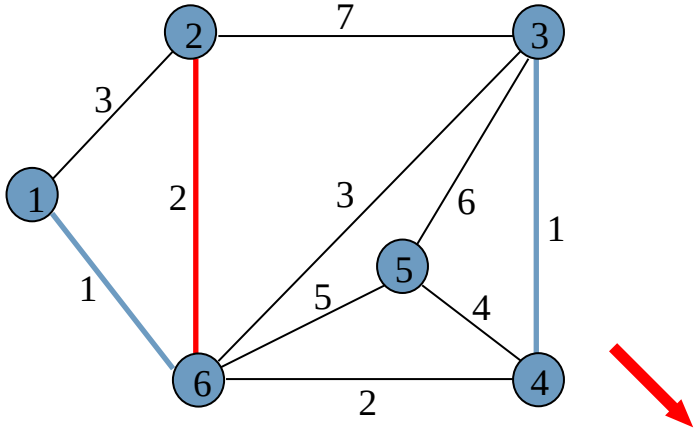
► Der Wald besteht nun noch aus **vier** Bäumen.



Vereinigungsschritt:

1. Kante $\mathbf{k} = (2,6)$ mit Gewicht 2 wird gewählt.
2. Die beiden Bäumen, die aus dem Knoten 2 und $\{1,6\}$ bestehen, werden zu einem Baum vereinigt.

► Der Wald besteht nun noch aus **drei** Bäumen.

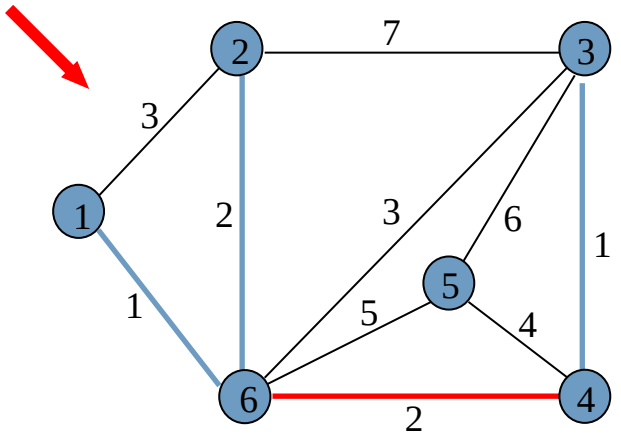


§3.5 Algorithmen von Prim und Kruskal

Vereinigungsschritt:

1. Kante $\mathbf{k} = (4,6)$ mit Gewicht 2 wird gewählt.
2. Die beiden Bäumen, die aus dem Knoten $\{3,4\}$ und $\{1,2,6\}$ bestehen, werden zu einem Baum vereinigt.

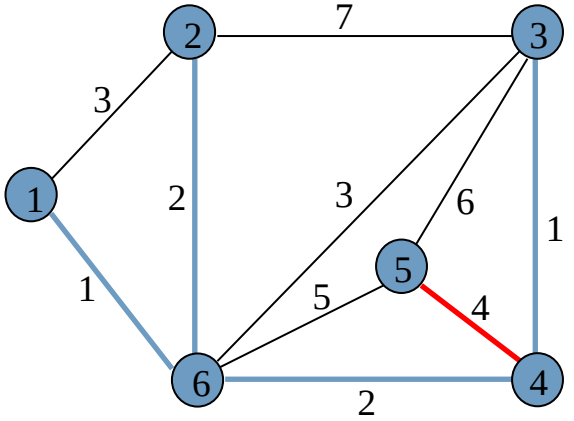
- Der Wald besteht nun noch aus **zwei** Bäumen.



Vereinigungsschritt:

1. Kante $\mathbf{k} = (4,5)$ mit Gewicht 4 wird gewählt.
2. Die beiden Bäumen, die aus dem Knoten 5 und $\{1,2,3,4,6\}$ bestehen, werden zu einem Baum vereinigt.

- Der Wald besteht nun noch aus **einem** Baum.
- Minimal aufspannender Baum gefunden.



§3.5 Algorithmen von Prim und Kruskal

Ausgabe

Eingabe

```
Tree minimalSpanningTree(Graph G) // G = (V,E)
{
    Set           forest(V);
(1) PriorityQueue edges (E);
    Tree          minTree=∅;
    while ( forest.size()!=1 && edges!=∅ ) {
(2)   (v,w)  = edges.delMin( );
(3)   Set t1 = forest.find (v);
(3)   Set t2 = forest.find (w);
      if ( t1!=t2 ) {
(4)      forest.union(t1,t2);
(5)      minTree.add(v,w);
      }
    }
    if ( forest.size()!=1 && edges==∅ ) return null;
    else                               return minTree;
}
```

Menge von Bäumen, die anfangs aus genau einem Knoten aus **V** bestehen.

Alle Kanten werden mit den Gewichten in einer Vorrangswarteschlange gespeichert.

Greedy-Schritt.

forest.find(v) liefert den Baum, in dem v enthalten ist.

Vereinige die beiden Bäume zu einem Baum.

Es gibt keinen aufspannenden Baum.

Bemerkungen

- Der Wald von Bäumen lässt sich effizient mit einer so genannten Union-Find-Struktur (Datentyp **`Set`**) darstellen, siehe Appendix:

  `find` (Zeile **`(3)`**): $O(\log n)$

  `union` (Zeile **`(4)`**): $O(1)$

- Eine Vorrangswarteschlange **`ProrityQueue`** lässt sich mit einem Heap implementieren:

  Aufbau einer **`ProrityQueue`** (Zeile **`(1)`**): $O(n)$

  `delMin` (Zeile **`(2)`**): $O(\log n)$

- Der minimal aufspannende Baum **`minTree`** lässt sich durch eine einfache Kanten-Liste realisieren:

  `add` (Zeile **`(5)`**): $O(1)$

Analyse

- Zeile `(1)` wird genau einmal ausgeführt.
- Im schlechtesten Fall wird **`delMin`** (Zeile `(2)`) und **`find`** (Zeilen `(3)`) und evtl. `(4)` und `(5)` $|E|$-mal ausgeführt:

$$T = O(|E| + |E|\,(\log|E| + \log|V|)) = O(|E|\log|E|).$$

Optimalitätsbeweise für Prim und Kruskal:

- Siehe [Kleinberg/Tardos: Algorithm Design, 2006]

Appendix: Heaps

- In einem binären Suchbaum gilt für einen Knoten k die Beziehung:

  `k.left.key < k.key < k.right.key.`

  Diese Eigenschaft ermöglicht eine schnelle Suche nach einem bestimmten Schlüssel.

- Sucht man jedoch den maximalen (minimalen) Schlüssel, ist für einen Knoten k die folgende Beziehung besser:

  `k.key > k.left.key, k.right.key.`

  Diese Eigenschaft heißt **(Max)-Heap-Eigenschaft**.

- Die Heap-Eigenschaft hat zur Folge, dass der Knoten mit dem größten Schlüssel in der Wurzel steht.

Appendix: Heaps

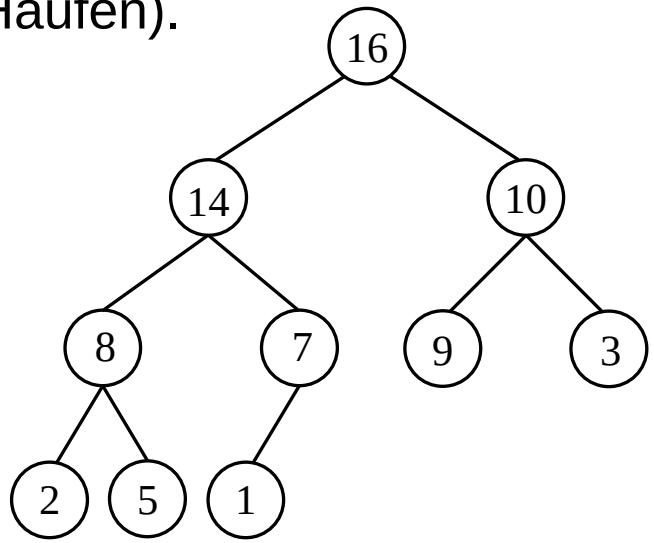
Definition

Ein binärer Baum,

1. der für jeden Knoten die (Max)-Heap-Eigenschaft erfüllt,
2. bei dem alle Schichten bis auf die letzte vollständig aufgefüllt sind und
3. die letzte Schicht linksbündig aufgefüllt ist,

heißt **Heap** (dt.: Halde, Haufen).

Beispiel



Appendix: Heaps

Eigenschaften

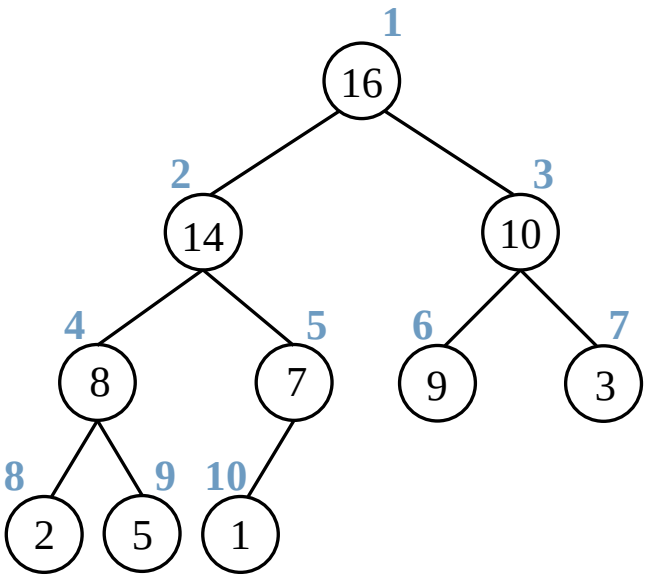
- In der Wurzel eines Max-Heaps steht der Knoten mit dem größten Schlüssel.
- In der Wurzel eines Min-Heaps steht der Knoten mit dem kleinsten Schlüssel.
- Weil ein Heap ein vollständiger binärer Baum ist, ist die Höhe eines Heaps mit n Einträgen $\Theta(\log n)$.

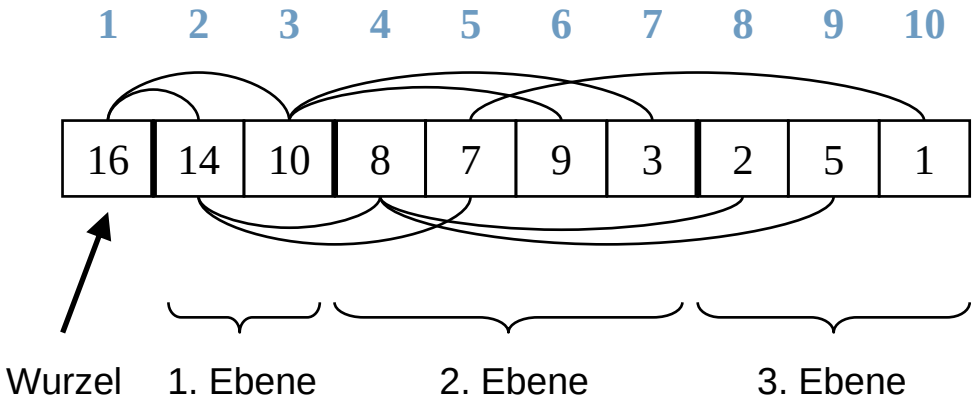
Appendix: Heaps

Bemerkung

- Ein Heap kann auch mit einem Array **`a[1..n]`** in-situ realisiert werden.
- Die Heap-Eigenschaft hat dann die Form

`a[i].key > a[2i].key, a[2i+1].key`





Konstruktion eines Heaps

- Ordne die n Datensätze in einem Array **`a[1..n]`** beliebig an.
  - Das entspricht einer beliebigen Anordnung in einem binären Baum mit den Eigenschaften 2. und 3.
- Anschließend muss noch die Heap-Eigenschaft hergestellt werden.
  - Jedes Blatt ist bereits ein einelementiger Heap.
  - Das Teilfeld **`a[(⌊n/2⌋+1)..n]`** enthält nur Blätter.
  - Die Heap-Eigenschaft kann dann mit einer Hilfsfunktion **`heapify`** von den Blättern zur Wurzel (bottom-up) hergestellt werden.
- ...

Appendix: Heaps

- ...
- Anschließend muss noch die Heap-Eigenschaft hergestellt werden.
  - Jedes Blatt ist bereits ein einelementiger Heap.
  - Das Teilfeld **`a[ ⌊n/2⌋+1 .. n]`** enthält nur Blätter.
  - Die Heap-Eigenschaft kann dann mit einer Hilfsfunktion **`heapify`** von den Blättern zur Wurzel (bottom-up) hergestellt werden.

```
void build_heap(Node[] a)
{
   for (int i = floor(length(a)/2); i > 0; i--) {
      heapify(a,i);
   }
}
```

Appendix: Heaps

- Die Hilfsfunktion **`heapify`** arbeitet im Baum von den Blättern zur Wurzel.
  - An einem zu bearbeitenden Knoten haben beide Kinder bereits die Heap-Eigenschaft.
  - Ist der Schlüssel größer als in beiden Kindknoten, ist die Heap-Eigenschaft erfüllt.
  - Anderenfalls muss der aktuelle Knoten
    a. mit dem Kind mit dem größeren Schlüssel vertauscht werden und
    b. in dem zugehörigen Teilbaum absinken, bis beide Nachfolger kleinere Schlüssel haben.

Appendix: Heaps

heapify

```
void heapify(Node[] a, int i)
{
   int max = i;
   int li  = 2*i;
   int re  = li+1;
   if (a[li].key > a[max].key) max = li;
   if (a[re].key > a[max].key) max = re;
   if (i != max) {
      swap(a[i],a[max]);
      heapify(a,max);
   }
}
```

```
// Achtung: Hier müsste
   noch getestet werden,
   ob li und re gültige
   Indizes sind!
```

Laufzeit der Konstruktion (`heapify`)

- Laufzeit von **`heapify`** auf einem Teilbaum der Größe n.
  - Bestimmung der Beziehungen von **`a[i]`**, **`a[le]`**, **`a[re]`**: $\Theta(1)$.
  - Ausführung von **`heapify`** auf einem Teilbaum.
    - Beide Teilbäume haben jeweils die maximale Größe $2n/3$, falls die letzte Ebene des Baumes genau halb gefüllt ist.
  - Laufzeit von **`heapify`**: $T(n) \leq T(2n/3) + \Theta(1)$
    - $T(n) = O(\log n)$.
  - Alternativ: Laufzeit von **`heapify`** für einen Knoten der Höhe h: $O(h)$.

Laufzeit der Konstruktion (`build_heap`)

- Laufzeit von **`build_heap`** für ein Feld der Größe n.
  - Obere Schranke: $O(n)$ Aufrufe von **`heapify`**
    - $O(n \log n)$.
  - Schärfere Schranke: Die Laufzeit von **`heapify`** variiert mit der Höhe des Teilbaums und ist meist klein.
    - Ein n-elementiger Heap hat die Höhe $\lfloor \log n \rfloor$.
    - Es gibt höchstens $\lceil n/2^{h+1} \rceil$ Knoten der Höhe h.
    - Gesamtlaufzeit:

$$O(h) \sum_{h=0}^{\lfloor \log n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil = O\left(n \sum_{h=0}^{\lfloor \log n \rfloor} \frac{h}{2^{h+1}} \right) = O\left(n \underbrace{\sum_{h=0}^{\infty} \frac{h}{2^{h+1}}}_{=2} \right) = O(n).$$

Appendix: Union-Find-Strukturen

Problem

- Effiziente Verwaltung einer Kollektion (Menge) von disjunkten Mengen

$$\texttt{col} = \{S_0, S_1, \ldots, S_{n-1}\}, \quad S_i \cap S_j = \emptyset \text{ für } i \neq j,$$

wobei folgende Operationen unterstützt werden:

`find(e)`: Liefert die Menge zurück, die **`e`** enthält.

`union(`S_1`,`S_2`)`: Vereinigt die beiden Mengen **``$S_1$``** und **``$S_2$``** aus der Kollektion zu einer neuen Menge.

- Es ist bei **`find`** und **`union`** gleichgültig, wie die Mengen benannt sind.
- Daher kann als Name der Menge ein beliebiges Element der Menge (**Repräsentant der Menge**) gewählt.
- **Typische Anwendung:**

```
col.union(col.find(x),col.find(y)).
```

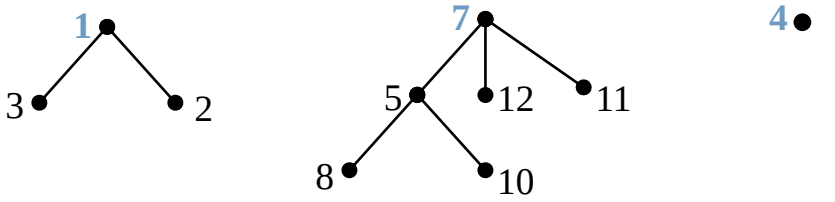
Appendix: Union-Find-Strukturen

Datenstruktur

- Verwalte die Mengen als Bäume.
- Die Bäume werden mit einem Elternfeld **`p`** dargestellt.
- Der Repräsentant einer Menge ist die Wurzel des Baumes.

Beispiel

- Kollektion von Mengen: **`col`**$= \{\{1,2,3\},\{5,7,8,10,11,12\},\{4\},\{6,9\}\}$
- Bäume:



Repräsentant der Menge

- Elternfeld:

e	1	2	3	4	5	6	7	8	9	10	11	12
`p[e]`	-	1	1	-	7	-	-	5	6	5	7	7

- **`p[e]`** gibt den Elternknoten zu **`e`** an.
- Die Wurzeln hat keinen Elternknoten.

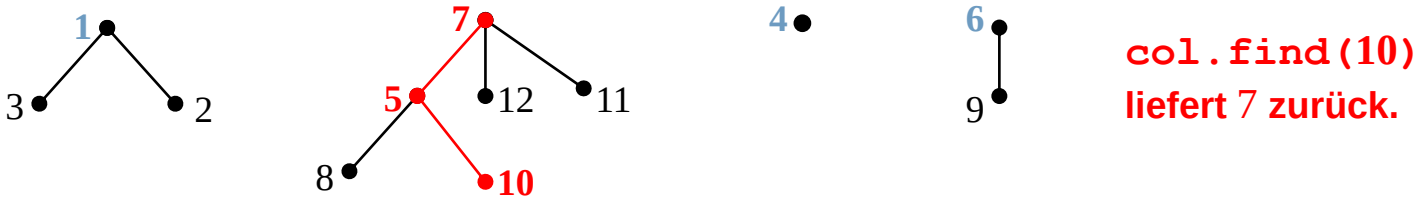
Appendix: Union-Find-Strukturen

Find-Algorithmus

```
int find(int e)
{
   if ( p[e]==undef. ) return e;
   else                return find(p[e]);
}
```

Beispiel

- Kollektion von Mengen: **col**= $\{\{1,2,3\},\{5,7,8,10,11,12\},\{4\},\{6,9\}\}$
- Bäume:



col.find(10)
liefert 7 zurück.

- Elternfeld:

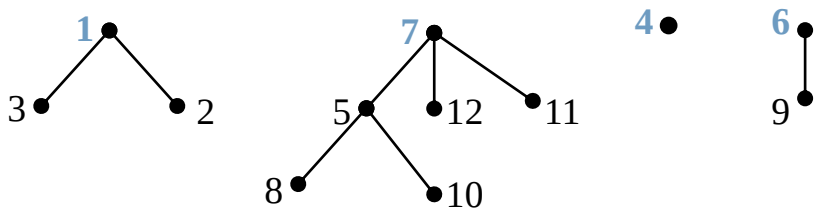
e	1	2	3	4	5	6	7	8	9	10	11	12
p[e]	-	1	1	-	7	-	-	5	6	5	7	7

Appendix: Union-Find-Strukturen

Union-Algorithmus

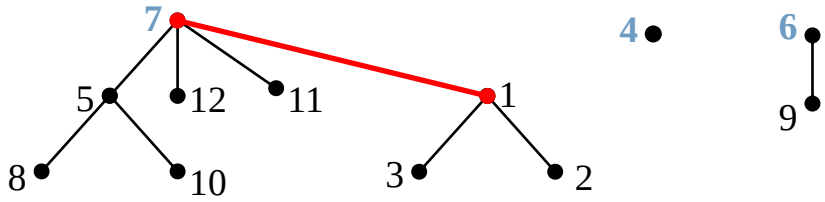
- Werden zwei Mengen vereinigt, wird der kleinere Baum als Kind der Wurzel des größeren Baumes eingehängt.

Beispiel



e	1	2	3	4	5	6	7	8	9	10	11	12
p[e]	-	1	1	-	7	-	-	5	6	5	7	7

`col.union(1,7)` vereinigt die Mengen $\{1,2,3\}$ und $\{5,7,8,10,11,12\}$:



e	1	2	3	4	5	6	7	8	9	10	11	12
p[e]	7	1	1	-	7	-	-	5	6	5	7	7

Appendix: Union-Find-Strukturen

- Es gibt zwei Möglichkeiten festzulegen, welcher Baum der Kleinere ist:
  - Baum $\mathbf{B_1}$ ist kleiner als Baum $\mathbf{B_2}$, falls die Höhe von $\mathbf{B_1}$ kleiner als die Höhe von $\mathbf{B_2}$ ist.
    - **`unionByHeight`**
  - Baum $\mathbf{B_1}$ ist kleiner als Baum $\mathbf{B_2}$, falls die Anzahl der Knoten von $\mathbf{B_1}$ kleiner ist als die Anzahl der Knoten von $\mathbf{B_2}$.
    - **`unionBySize`**
- Die Höhen- bzw. Größeninformation für ein Baum mit Wurzel **`e`** lässt sich bei **`p[e]`** als negative Zahl abspeichern.

Appendix: Union-Find-Strukturen

Union-By-Height

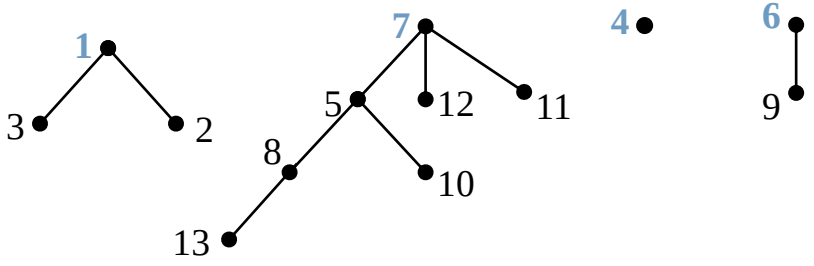
```
void unionByHeight(int s1, int s2)
{
   if ( -p[s1] < -p[s2] ) p[s1] = s2;
   else {
      if ( p[s1]==p[s2] ) p[s1]--;
      p[s2] = s1;
   }
}
```

Union-By-Size

```
void unionBySize(int s1, int s2)
{
   if ( -p[s1] < -p[s2] ) {
      p[s2] += p[s1]; p[s1] = s2;
   } else {
      p[s1] += p[s2]; p[s2] = s1;
   }
}
```

Appendix: Union-Find-Strukturen

Beispiel für `unionByHeight`:

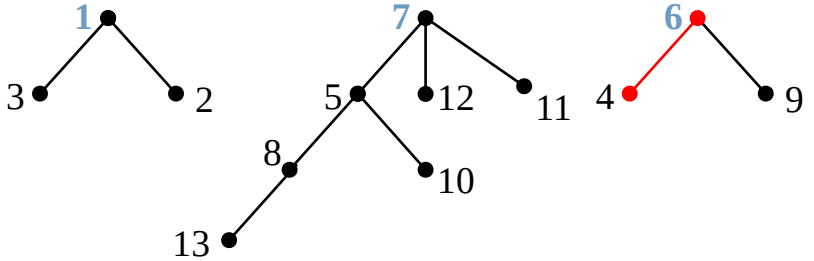


e	1	2	3	4	5	6	7	8	9	10	11	12	13
p[e]	-1	1	1	0	7	-1	-3	5	6	5	7	7	8

Wurzeln

Höheninformation

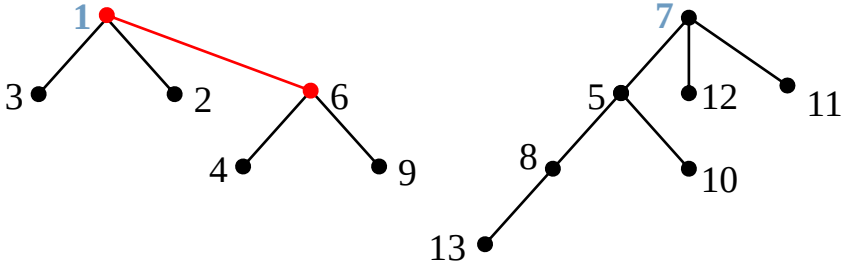
`unionByHeight(4,6)`



e	1	2	3	4	5	6	7	8	9	10	11	12	13
p[e]	-1	1	1	6	7	-1	-3	5	6	5	7	7	8

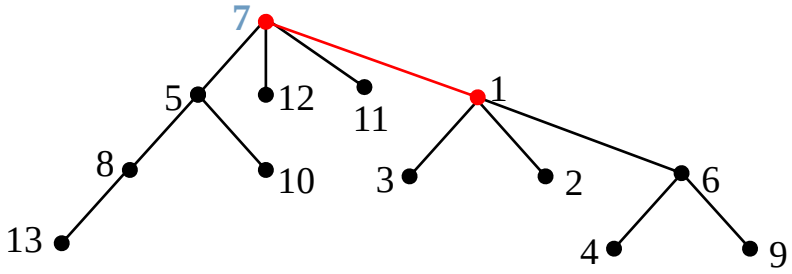
Appendix: Union-Find-Strukturen

unionByHeight(1,6)



e	1	2	3	4	5	6	7	8	9	10	11	12	13
p[e]	-2	1	1	6	7	1	-3	5	6	5	7	7	8

unionByHeight(1,7)



e	1	2	3	4	5	6	7	8	9	10	11	12	13
p[e]	7	1	1	6	7	1	-3	5	6	5	7	7	8

Appendix: Union-Find-Strukturen

Analyse

- **`union`**: $T(n) = O(1)$
- **`find`**: $T(n) = O(\log n)$